

# Heart Disease Prediction

## Using Machine Learning

CSS 324: Introduction to Machine Learning  
Final Project Report

| Team Member       | Role                                                          |
|-------------------|---------------------------------------------------------------|
| Nazerke Kenges    | Exploratory Data Analysis & Report Writing & Interactive Demo |
| Alikhan Kulibayev | Model Training & Hyperparameter Tuning                        |
| Zhuldyz Leken     | Data Preprocessing & Feature Engineering                      |
| Aruzhan Anarbek   | Error Analysis & Report Writing                               |
| Aidana Alibek     | Model Training & Evaluation                                   |

Dataset: Heart Disease Dataset (Kaggle)

## Contents

---

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Topic &amp; Motivation</b>                                           | <b>2</b>  |
| <b>2</b> | <b>Dataset &amp; Preparation</b>                                        | <b>2</b>  |
| 2.1      | Dataset Source . . . . .                                                | 2         |
| 2.2      | Data Cleaning . . . . .                                                 | 2         |
| 2.3      | Missing Value Imputation . . . . .                                      | 3         |
| 2.4      | Outlier Handling . . . . .                                              | 3         |
| 2.5      | Feature Engineering . . . . .                                           | 3         |
| <b>3</b> | <b>Research &amp; Related Work</b>                                      | <b>3</b>  |
| 3.1      | Heart Disease Prediction Using Machine Learning . . . . .               | 3         |
| 3.2      | Effective Heart Disease Prediction Using Hybrid ML Techniques . . . . . | 4         |
| 3.3      | Heart Disease Prediction Using Machine Learning Techniques . . . . .    | 4         |
| 3.4      | Key Takeaway from the Literature . . . . .                              | 4         |
| <b>4</b> | <b>Exploratory Data Analysis (EDA)</b>                                  | <b>4</b>  |
| 4.1      | Statistical Summary . . . . .                                           | 4         |
| 4.2      | Class Distribution . . . . .                                            | 4         |
| 4.3      | Key Visual Findings . . . . .                                           | 4         |
| <b>5</b> | <b>Machine Learning Experiments</b>                                     | <b>6</b>  |
| 5.1      | Experimental Setup . . . . .                                            | 6         |
| 5.2      | Model 1: Logistic Regression . . . . .                                  | 7         |
| 5.3      | Model 2: Decision Tree . . . . .                                        | 7         |
| 5.4      | Model 3: Random Forest . . . . .                                        | 7         |
| 5.5      | Model Comparison . . . . .                                              | 7         |
| 5.6      | Final Model Selection . . . . .                                         | 7         |
| <b>6</b> | <b>Error Analysis &amp; Limitations</b>                                 | <b>8</b>  |
| 6.1      | Confusion Matrix (Logistic Regression) . . . . .                        | 8         |
| 6.2      | Key Limitations . . . . .                                               | 8         |
| 6.3      | Discussion . . . . .                                                    | 8         |
| <b>7</b> | <b>Final Model &amp; Interactive Demo</b>                               | <b>9</b>  |
| 7.1      | Model Description . . . . .                                             | 9         |
| 7.2      | Interactive Jupyter Demo . . . . .                                      | 9         |
| <b>8</b> | <b>Conclusions</b>                                                      | <b>9</b>  |
|          | <b>References</b>                                                       | <b>10</b> |

## 1. Topic & Motivation

---

Cardiovascular disease remains one of the leading causes of death worldwide. Early identification of individuals at high risk of heart stroke can enable timely medical intervention and significantly reduce mortality. Traditional diagnostic methods rely heavily on clinical tests and physician expertise, which may not always be accessible in resource-limited settings.

Machine learning offers a promising complement to traditional diagnostics: given a set of patient attributes (age, blood pressure, smoking habits, glucose levels, etc.), a trained model can automatically flag high-risk individuals for further examination. This project investigates whether standard classification algorithms can reliably predict the onset of heart stroke from tabular clinical data.

**Research question:** Can we build a machine learning model that identifies patients at risk of heart stroke from demographic and clinical features, and which algorithm performs best on this task?

The problem is framed as a binary classification task, where the target variable is `heart_stroke` (1 = stroke occurred, 0 = no stroke).

## 2. Dataset & Preparation

---

### 2.1 Dataset Source

- **Source:** Kaggle – Heart Disease dataset
- **File:** `heart_disease.csv`
- **Target variable:** `Heart_stroke` (binary: Yes / No)

The raw dataset contains patient records with demographic, lifestyle, and clinical features relevant to cardiovascular risk assessment.

### 2.2 Data Cleaning

The first step was to standardise column names and encode categorical variables:

```
df = pd.read_csv('heart_disease.csv')
df = df.rename(columns={'Heart_stroke': 'heart_stroke'})
df.columns = df.columns.str.strip().str.lower()

# Binary encoding
df['heart_stroke'] = df['heart_stroke'].astype(str).str.strip().str.lower().map({'yes': 1, 'no': 0})
df['gender'] = df['gender'].astype(str).str.strip().str.lower().map({'male': 1, 'female': 0})
df['prevalentstroke'] = df['prevalentstroke'].astype(str).str.strip().str.lower().map({'yes': 1, 'no': 0})

# Ordinal encoding for education
edu_order = {'uneducated': 0, 'primaryschool': 1, 'graduate': 2, 'postgraduate': 3}
df['education'] = df['education'].astype(str).str.strip().str.lower().map(edu_order)
```

## 2.3 Missing Value Imputation

Several columns contained missing values, which were handled using domain-aware imputation strategies rather than a single global fill:

- **cigsperday:** Median imputed within each `currentsmoker` group (stratified imputation).
- **bpmeds:** Filled with the mode (most common value = 0).
- **totchol, bmi, heartrate:** Global median imputation.
- **glucose:** Median imputed within each `diabetes` group.
- **education:** Missing values set to 0 (uneducated, the baseline).

After imputation, the dataset had zero remaining null values.

## 2.4 Outlier Handling

Outliers in continuous features were capped using the IQR method (Winsorization at the  $1.5 \times \text{IQR}$  boundary), applied to: `cigsperday`, `totchol`, `sysbp`, `diabp`, `bmi`, `heartrate`, and `glucose`.

## 2.5 Feature Engineering

Three new clinically motivated features were created:

- **pulse\_pressure:** Systolic BP – Diastolic BP. A known indicator of arterial stiffness and cardiovascular risk.
- **heavy\_smoker:** Binary flag set to 1 if `cigsperday` > 20 (more than one pack per day).
- **age\_group:** Ordinal binning of age into four categories: <40, 40–50, 50–60, 60+ (encoded as 0–3).

The final cleaned dataset was saved as `heart_disease_cleaned.csv` for model training.

# 3. Research & Related Work

---

Before conducting our experiments, we reviewed three relevant papers from the literature on heart disease prediction with machine learning.

## 3.1 Heart Disease Prediction Using Machine Learning

*Sashank Yadav, Aman Singh, Veena Jadhav, Dr. Rohini Jadhav*

This paper benchmarked seven classification algorithms (Naïve Bayes, Decision Tree, Logistic Regression, KNN, SVM, Gradient Boosting, and Random Forest) on the UCI heart disease dataset. KNN achieved the highest accuracy at 85.18%. The authors also deployed their final model as a Flask web application using a serialised `.pkl` file, which directly inspired our own approach of saving the final model with `joblib` for the interactive Jupyter demo.

### 3.2 Effective Heart Disease Prediction Using Hybrid ML Techniques

*Mohan, Thirumalai & Srivastava — IEEE*

This paper proposed a hybrid model (HRFLM) combining Random Forest with a linear model, achieving 88.7% accuracy. Their emphasis on feature selection to boost performance directly influenced our decision to engineer domain-specific features such as `pulse_pressure` and `age_group`.

### 3.3 Heart Disease Prediction Using Machine Learning Techniques

*Apurv Garg, Bhartendu Sharma & Rijwan Khan — IOP Publishing*

This paper compared KNN (86.9% accuracy) and Random Forest (82.0% accuracy) for heart disease classification. Their finding that tree-based ensemble methods generalise well on medical data supported our choice to include Random Forest as one of the three evaluated models.

### 3.4 Key Takeaway from the Literature

All three papers consistently showed that ensemble methods — particularly Random Forest — perform strongly on cardiovascular prediction tasks, and that features related to blood pressure, age, and smoking habits are clinically meaningful predictors. These findings validated our model selection strategy and feature engineering decisions.

## 4. Exploratory Data Analysis (EDA)

### 4.1 Statistical Summary

A statistical summary (`df.describe()`) was computed for all numeric features to inspect central tendency, spread, and potential anomalies before and after preprocessing.

### 4.2 Class Distribution

The target variable (`heart_stroke`) is highly imbalanced:

| Class         | Count (approx.) | Proportion |
|---------------|-----------------|------------|
| No Stroke (0) | ~719            | ~85%       |
| Stroke (1)    | ~129            | ~15%       |

Table 1: Class distribution of the target variable.

This imbalance was a central challenge addressed in all three models using the `class_weight='balanced'` parameter.

### 4.3 Key Visual Findings

The following visualisations were produced during EDA:

1. **Target distribution** (`countplot`): Confirmed the severe class imbalance described above.

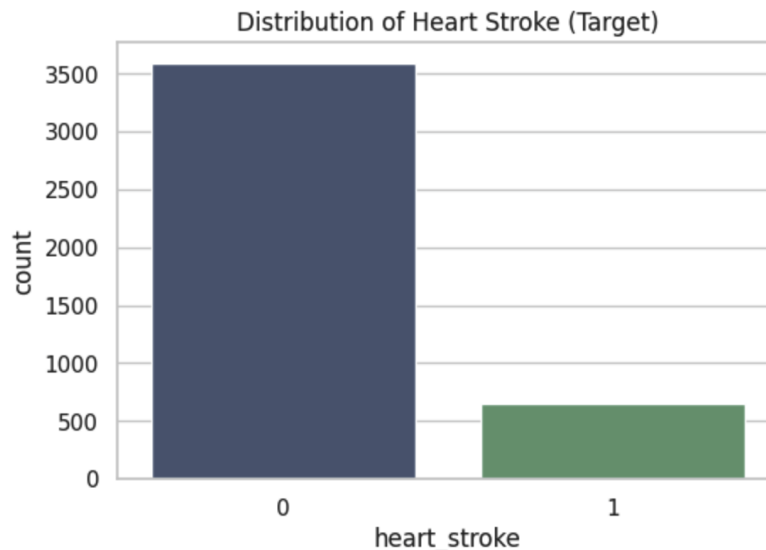
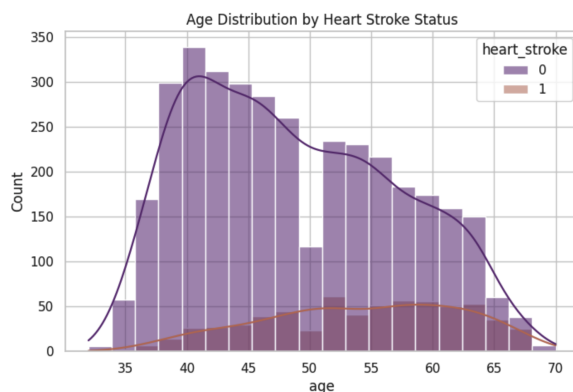
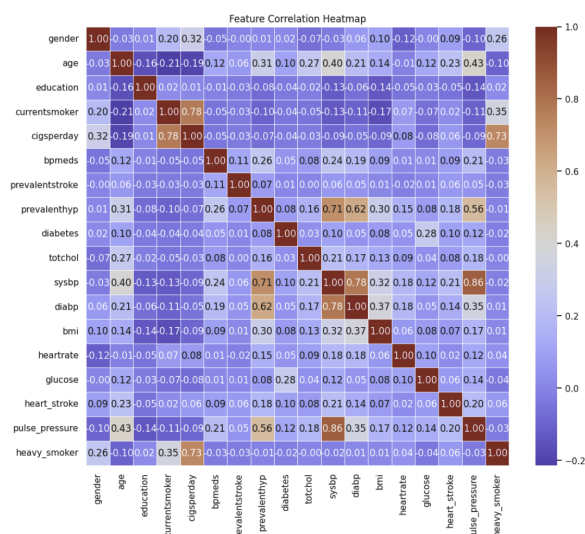


Figure 1: Distribution of Heart Stroke (Target Variable)

2. **Age distribution by stroke status** (histplot with KDE): Stroke cases are concentrated in older patients (roughly 55+), suggesting age is a strong predictor.
3. **Correlation heatmap**: Features with the highest positive correlation to `heart_stroke` include `age`, `sysbp`, `pulse_pressure`, and `glucose`. `cigsperday` and `prevalentstroke` also show moderate correlation.



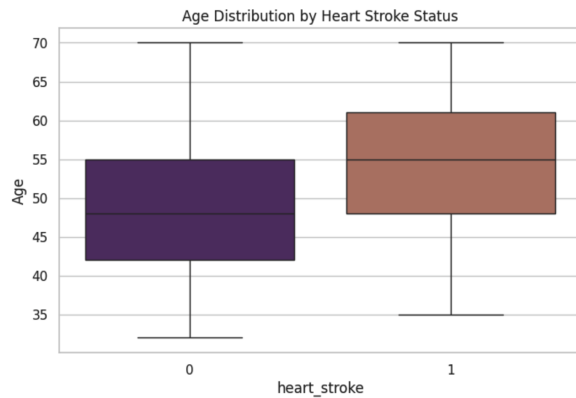
(a) Age Distribution by Stroke Status



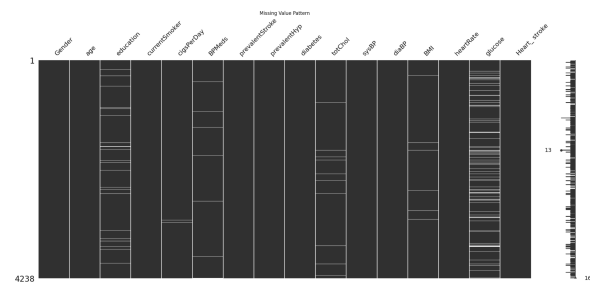
(b) Feature Correlation Heatmap

Figure 2: EDA Visualizations

4. **Age boxplot by stroke status**: Patients who experienced a stroke have a visibly higher median age.
5. **Missing value matrix** (`missingno`): Revealed scattered missingness in `glucose`, `bmi`, `totchol`, `heartrate`, `cigsperday`, `bpmeds`, and `education`.
6. **Outlier boxplots**: Confirmed extreme outliers in `glucose`, `totchol`, and `sysbp` that were subsequently clipped via IQR Winsorization.



(a) Age boxplot by stroke status. Stroke patients have a visibly higher median age.



(b) Missing value matrix. Scattered missingness in `glucose`, `bmi`, `totchol`, `heartrate`, `cigasperday`, `bpmeds`, and `education`.

Figure 3: Age distribution and missing data pattern.

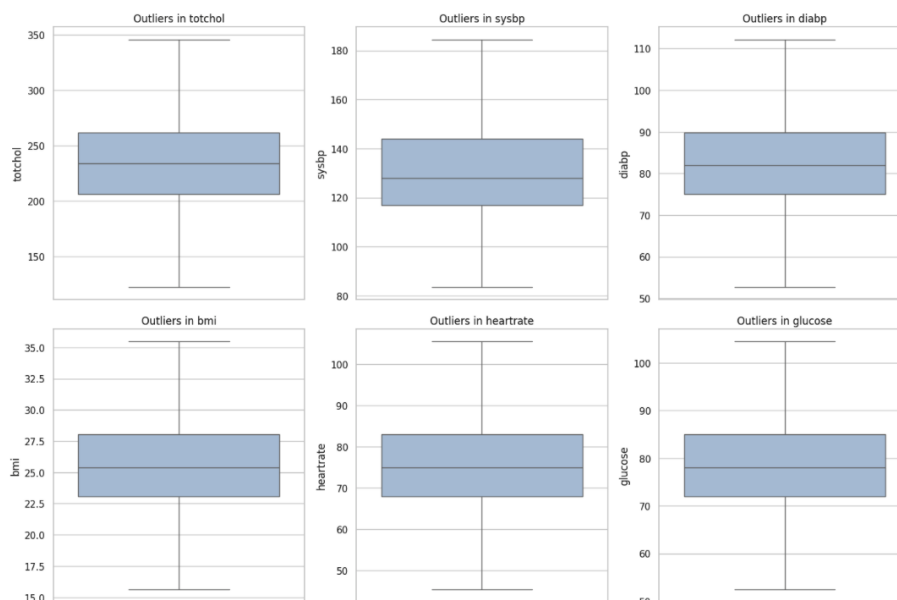


Figure 4: Outlier boxplots for continuous features. Extreme outliers confirmed in `glucose`, `totchol`, and `sysbp`, subsequently clipped via IQR Winsorization.

## 5. Machine Learning Experiments

### 5.1 Experimental Setup

The cleaned dataset was split 80/20 into training and test sets using stratified sampling to preserve the class ratio:

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

A `SimpleImputer` (median strategy) and `StandardScaler` were applied inside each

pipeline. The evaluation metric prioritised for model selection is the **F1-score** on the minority class (stroke = 1), because class imbalance makes raw accuracy misleading.

## 5.2 Model 1: Logistic Regression

A logistic regression classifier with `class_weight = 'balanced'` and `max_iter = 1000` was trained inside a scikit-learn **Pipeline** that includes imputation and scaling.

**Rationale:** Logistic regression is a strong baseline for binary classification. It is interpretable, robust to irrelevant features after scaling, and handles class imbalance well with the balanced weight setting.

## 5.3 Model 2: Decision Tree

A decision tree classifier with `class_weight = 'balanced'` was tuned via 5-fold cross-validated grid search over:

- `max_depth`: {3, 5, 7, 10}
- `min_samples_split`: {2, 5, 10}

**Rationale:** Decision trees are non-linear and interpretable. Grid search over depth and split size prevents overfitting while maximising the F1-score on validation folds.

## 5.4 Model 3: Random Forest

A random forest with `class_weight='balanced'` was tuned via 5-fold cross-validated grid search over:

- `n_estimators`: {100, 200}
- `max_depth`: {5, 10, None}
- `min_samples_split`: {2, 5}

**Rationale:** Random forests are an ensemble of decision trees that reduce variance through bagging. They are consistently strong on tabular medical data, as confirmed by the literature review.

## 5.5 Model Comparison

| Model               | Accuracy | Precision | Recall | F1-Score     |
|---------------------|----------|-----------|--------|--------------|
| Logistic Regression | 0.665    | 0.244     | 0.574  | <b>0.343</b> |
| Random Forest       | 0.671    | 0.232     | 0.504  | 0.318        |
| Decision Tree       | 0.610    | 0.206     | 0.550  | 0.300        |

Table 2: Performance comparison of the three models on the test set (sorted by F1-score).

## 5.6 Final Model Selection

**Logistic Regression** was selected as the final model based on the highest F1-score (0.343). Although Random Forest achieves marginally higher accuracy (0.671 vs. 0.665), accuracy



is not the correct metric here due to class imbalance. Logistic Regression achieves better recall (0.574 vs. 0.504), meaning it correctly identifies more true stroke cases — which is the clinically critical outcome.

The final model was serialised with `joblib`:

```
import joblib
joblib.dump(rf_grid, 'final_model.pkl')
```

## 6. Error Analysis & Limitations

### 6.1 Confusion Matrix (Logistic Regression)

|                   | Predicted: No Stroke | Predicted: Stroke |
|-------------------|----------------------|-------------------|
| Actual: No Stroke | 490 (TN)             | 229 (FP)          |
| Actual: Stroke    | 55 (FN)              | 74 (TP)           |

Table 3: Confusion matrix for the final Logistic Regression model.

- The model correctly identifies **74 out of 129** stroke cases (57.4% recall).
- **229 false positives**: patients flagged as high-risk who did not experience a stroke. In a clinical screening context this leads to unnecessary follow-up costs.
- **55 false negatives**: actual stroke cases that were missed. These are the more dangerous errors in a medical setting.

### 6.2 Key Limitations

1. **Severe class imbalance.** The dataset is approximately 85% non-stroke, 15% stroke. Despite `class_weight = 'balanced'`, all models achieve low precision ( $\approx 0.21$ – $0.24$ ) on the stroke class.
2. **Low precision on the stroke class.** A precision of  $\approx 0.24$  means roughly 3 out of 4 positive predictions are false alarms. In a clinical context this could cause undue patient anxiety.
3. **Similar performance across models.** The narrow F1-score range (0.30–0.34) suggests the bottleneck is data quality and quantity, not the choice of algorithm.
4. **Small dataset.** The test set contains only 848 samples, which is modest for a medical prediction task. A larger, more diverse dataset would improve generalisation.
5. **Missing clinically relevant features.** Family history of heart disease, medication history, and stress levels are absent, which likely limits predictive power.

### 6.3 Discussion

In a medical screening scenario, **recall is the priority metric**: it is far better to over-predict stroke risk and perform follow-up tests than to miss a real case. Given this, Logistic Regression is the most appropriate model because it achieves the highest recall (0.574) and F1-score (0.343). Future work should explore SMOTE oversampling, cost-sensitive

learning, or gradient boosting methods (e.g., XGBoost, LightGBM) to further improve minority-class performance.

## 7. Final Model & Interactive Demo

---

### 7.1 Model Description

The final deployed model is a **Logistic Regression** classifier trained on the cleaned and feature-engineered heart disease dataset. Input features required by the model are:

gender, age, education, currentsmoker, cigspersday, bpmeds,  
prevalentstroke, prevalentthyp, diabetes, totchol, sysbp, diabbp,  
bmi, heartrate, glucose, pulse\_pressure, heavy\_smoker, age\_group

### 7.2 Interactive Jupyter Demo

The last cell of `finalprojectML.ipynb` provides a fully interactive widget powered by `ipywidgets`. Users can adjust sliders and dropdowns for each input feature and click **Predict** to receive an instant risk assessment:

- **HIGH RISK** of heart stroke (with predicted probability)
- **LOW RISK** of heart stroke (with predicted probability)

The model is loaded from `final_model.pkl` using `joblib`, and the three engineered features (`pulse_pressure`, `heavy_smoker`, `age_group`) are computed automatically from the user's inputs before passing to the model.

## 8. Conclusions

---

This project demonstrated the application of machine learning to heart stroke risk prediction using a real-world clinical dataset from Kaggle. The main findings are:

1. **Logistic Regression outperforms** Decision Tree and Random Forest on this specific task when evaluated by F1-score, primarily because it achieves higher recall on the minority (stroke) class.
2. **Feature engineering adds value.** The three new features (`pulse_pressure`, `heavy_smoker`, `age_group`) encode clinically meaningful signals that improve model learning.
3. **Class imbalance is the key challenge.** All models struggle with low precision on the stroke class. Future work should apply advanced resampling techniques (SMOTE, ADASYN) or ensemble methods better suited to imbalanced data.
4. **Data quality matters more than algorithm choice.** The narrow performance gap between models suggests that a larger, cleaner, and more feature-rich dataset would yield greater improvements than further algorithm tuning.
5. **Interactive demo validates usability.** The `ipywidgets` demo allows non-technical users to explore stroke risk factors in real time, demonstrating the practical utility of the trained model.

The project repository and notebook (`finalprojectML.ipynb`) contain all reproducible code, data preprocessing steps, trained model file (`final_model.pkl`), and the interactive prediction demo.

## References

---

- [1] S. Yadav, A. Singh, V. Jadhav, R. Jadhav. *Heart Disease Prediction Using Machine Learning*. International Journal of Innovative Technology and Exploring Engineering (IJITEE), 2019.
- [2] S. Mohan, C. Thirumalai, G. Srivastava. *Effective Heart Disease Prediction Using Hybrid Machine Learning Techniques*. IEEE Access, Vol. 7, pp. 81542–81554, 2019.
- [3] A. Garg, B. Sharma, R. Khan. *Heart Disease Prediction Using Machine Learning Techniques*. IOP Conference Series: Materials Science and Engineering, Vol. 1022, 2021.